

به نام خدا

گزارش فاز اول پروژه

اعضای گروه :

زهرا علی محمدی

نگار شفیعی

فاطمه سمساری

موضوع پروژه : تشخیص نفوذ در سیستم های سایبری با استفاده از یادگیری ماشین
(مسیر دو - یادگیری ماشین کلاسیک صنعتی)

تعریف مسئله :

هدف این پروژه طراحی یک مدل یادگیری ماشین برای تشخیص نفوذ سایبری بر اساس داده های ترافیک شبکه و رفتار کاربران است. این مسئله به صورت طبقه بندی دودویی نظارت شده (Supervised Binary Classification) تعریف شده است، به طوری که هر نشست دسترسی به سیستم در یکی از دو دسته زیر قرار می گیرد:

- رفتار عادی (Normal)
- فعالیت مخرب (Malicious / Attack)

ورودی ها :

ورودی مدل شامل مجموعه ای از ویژگی های شبکه ای و رفتاری کاربران است:
ویژگی های شبکه ای:

- اندازه بسته شبکه
- نوع پروتکل
- نوع رمزنگاری

ویژگی های رفتاری:

- تعداد تلاش های ورود
- تعداد ورودهای ناموفق
- مدت زمان نشست
- دسترسی در زمان غیرعادی
- امتیاز اعتبار IP
- نوع مرورگر

خروجی مدل :

خروجی مدل یک برچسب دودویی است:

- 0 → رفتار عادی
 - 1 → فعالیت مخرب
- همچنین مدل می‌تواند احتمال وقوع حمله را نیز پیش‌بینی کند.

معیار موفقیت :

با توجه به ماهیت امنیتی مسئله، تنها دقت (Accuracy) معیار مناسبی نیست، بنابراین از معیارهای زیر استفاده می‌شود:

- F1-score (معیار اصلی)
- Recall (برای کاهش عدم شناسایی حملات)
- ROC-AUC (برای سنجش توان تفکیک مدل)

معرفی و تثبیت دیتاست :

در این پروژه از مجموعه داده‌ای با عنوان Cybersecurity Intrusion Detection Dataset استفاده شده است.

مشخصات دیتاست:

- نوع داده: جدولی (Tabular)
- فرمت فایل: CSV
- تعداد نمونه‌ها: 9537
- نام فایل: data/final/cybersecurity_intrusion_data_v1.csv
- متغیر هدف: attack_detected

نسخه v1 دیتاست به عنوان نسخه ثابت (Freeze) در نظر گرفته شده است و هیچ‌گونه تغییری مستقیماً روی آن اعمال نشده است. تمامی مراحل پیش‌پردازش و مهندسی ویژگی روی نسخه‌های جداگانه انجام شده‌اند.

ویژگی	نوع داده
network_packet_size	عددی
protocol_type	دسته ای
encryption_used	دسته ای
login_attempts	عددی
failed_logins	عددی
session_duration	عددی
unusual_time_access	دودویی
ip_reputation_score	عددی
browser_type	دسته ای
attack_detected	دودویی

EDA

نمودار 1 - Class distribution :

توزیع کلاس‌ها نشان می‌دهد که دیتاست دارای نسبت نسبتاً متعادل بین نمونه‌های نرمال و حمله است. این موضوع اهمیت استفاده از معیارهایی مانند Recall و score-F1 را به جای صرفاً Accuracy نشان می‌دهد.

نمودار ۲ — Missing Values:

بررسی مقادیر گم‌شده نشان داد که (مثلاً) داده‌ها فاقد مقدار گم‌شده هستند / یا برخی ستون‌ها دارای درصد کمی از داده‌های گم‌شده هستند. بر این اساس، در مرحله preprocessing از روش مناسب (مثلاً حذف یا جایگزینی) استفاده شد.

نمودار ۳ — توزیع ویژگی‌های عددی مهم :

بررسی توزیع ویژگی‌های عددی نشان می‌دهد که برخی ویژگی‌ها دارای توزیع نامتقارن (skewed) هستند. به طور خاص، تعداد login_attempts و failed_logins در نمونه‌های حمله تمایل به مقادیر بالاتر دارد. این ویژگی‌ها می‌توانند نقش مهمی در تشخیص رفتارهای مشکوک داشته باشند.

نمودار ۴ — مقایسه Normal vs Attack برای یک ویژگی کلیدی :

مقایسه توزیع ویژگی‌ها بین نمونه‌های نرمال و حمله نشان می‌دهد که رفتار کاربران در حملات سایبری تفاوت محسوسی با رفتار عادی دارد. برای مثال، تعداد failed_logins در حملات به طور قابل توجهی بیشتر است. این موضوع نشان می‌دهد که مدل‌های یادگیری ماشین قادر به یادگیری این الگوهای رفتاری خواهند بود.

نمودار ۵ — Correlation Heatmap :

بررسی همبستگی ویژگی‌ها نشان می‌دهد که برخی ویژگی‌ها با یکدیگر همبستگی دارند، اما همبستگی بسیار بالا (multicollinearity شدید) مشاهده نمی‌شود. این موضوع احتمال overfitting ناشی از وابستگی شدید ویژگی‌ها را کاهش می‌دهد.

نمودار ۶ — Outlier Analysis:

تحلیل نقاط پرت نشان می‌دهد که برخی مقادیر بسیار بزرگ در ویژگی‌های شبکه وجود دارد که می‌تواند نشان‌دهنده رفتار غیرعادی یا حملات خاص باشد. به دلیل استفاده از مدل‌های درختی در این پروژه، این نقاط پرت تأثیر منفی جدی بر مدل نخواهند داشت.

نتایج حاصل از EDA :

دیتاست دارای (متعادل / کمی نامتعادل) بودن کلاس‌ها است. ویژگی‌هایی مانند failed_logins و login_attempts ارتباط مستقیم با وقوع حمله دارند. توزیع برخی ویژگی‌های عددی دارای skewness است. مقادیر گم‌شده (وجود ندارد / بسیار کم است). برخی ویژگی‌ها رفتار متفاوتی بین کلاس نرمال و حمله نشان می‌دهند. دیتاست برای مسئله طبقه‌بندی دودویی مناسب است.

نتایج تحلیل اکتشافی نشان می‌دهد که دیتاست انتخاب‌شده دارای الگوهای مشخصی برای تفکیک رفتارهای نرمال و مخرب است. این تحلیل مبنای تصمیمات مرحله preprocessing و انتخاب مدل‌های مبتنی بر درخت در مراحل بعدی پروژه قرار گرفت.

Data preprocessing

پیش‌پردازش داده‌ها (Data Preprocessing) :

پس از انجام تحلیل اکتشافی داده‌ها (EDA)، مرحله پیش‌پردازش به منظور آماده‌سازی داده‌ها برای مدل‌های یادگیری ماشین اجرا شد. هدف این مرحله، پاک‌سازی داده‌ها، تبدیل ویژگی‌ها به فرم مناسب، و جلوگیری از نشت اطلاعات (data leakage) بود.

مدیریت مقادیر گمشده (Handling Missing Values) :

در این دیتاست، بررسی‌ها نشان داد که:
مقادیر گمشده وجود نداشت / یا درصد آن بسیار ناچیز بود.
در صورت وجود مقدار گمشده، از روش‌های مناسب مانند جایگزینی با مقدار میانگین (برای ویژگی‌های عددی) یا مد (برای ویژگی‌های دسته‌ای) استفاده شد.
این مرحله باعث افزایش پایداری مدل و جلوگیری از بروز خطا در فرآیند آموزش شد.

تبدیل ویژگی‌های دسته‌ای (Categorical Encoding) :

ویژگی‌های زیر دارای ماهیت دسته‌ای بودند:

protocol_type

encryption_used

browser_type

برای تبدیل این ویژگی‌ها به فرم قابل استفاده توسط مدل، از روش Encoding مناسب (مانند Hot En--One coding) استفاده شد.

این تبدیل باعث شد:

مدل بتواند از اطلاعات دسته‌ای استفاده کند

هیچ ترتیب مصنوعی و اشتباهی بین دسته‌ها ایجاد نشود

تبدیل ویژگی‌های دسته‌ای (Categorical Encoding) :

ویژگی‌های عددی مانند:

network_packet_size

session_duration

login_attempts

failed_logins

ip_reputation_score

در صورت نیاز نرمال‌سازی یا استانداردسازی شدند.

هرچند مدل‌های درختی (مانند Random Forest و XGBoost) به مقیاس ویژگی‌ها حساس نیستند، اما انجام استانداردسازی باعث:

یکنواختی پردازش داده و سازگاری با مدل‌های احتمالی دیگر در مراحل بعدی شد.

تقسیم داده‌ها (Train / Validation / Test Split) :

برای جلوگیری از نشت اطلاعات و ارزیابی منصفانه مدل، داده‌ها به سه بخش تقسیم شدند:

70٪ داده برای آموزش (Train)

15٪ برای اعتبارسنجی (Validation)

15٪ برای آزمون نهایی (Test)

تقسیم‌بندی با استفاده از پارامتر random_state ثابت انجام شد تا نتایج قابل بازتولید (reproducible) باشند. همچنین از Stratified Split استفاده شد تا نسبت کلاس‌ها در هر بخش حفظ شود.

ذخیره‌سازی داده‌های پردازش‌شده :

پس از انجام مراحل پیش‌پردازش، داده‌ها در مسیر زیر ذخیره شدند:

data/processed/

این کار باعث شد:

مرحله مدل‌سازی مستقل از preprocessing اجرا شود و تمام اعضای تیم بتوانند از نسخه یکسان داده‌ها استفاده کنند و بازتولید نتایج در مراحل بعدی تضمین شود .

جمع‌بندی بخش Preprocessing :

مرحله پیش‌پردازش نقش کلیدی در آماده‌سازی داده‌ها برای مدل‌سازی داشت. با انجام تبدیل‌های مناسب، حذف یا مدیریت مقادیر گمشده، و تقسیم‌بندی اصولی داده‌ها، زیرساختی پایدار برای آموزش مدل‌های طبقه‌بندی در مراحل بعدی پروژه فراهم شد.

Baseline

مدل پایه (Baseline Model) – Random Forest هدف از Baseline

در فاز اول پروژه، هدف اصلی از ساخت مدل پایه (Baseline) ایجاد یک نقطه‌ی مرجع قابل اعتماد برای مقایسه با مدل‌های پیشرفته‌تر در فازهای بعدی است. Baseline باید: ساده و سریع اجرا شود، نیاز به تنظیمات پیچیده نداشته باشد، روی داده‌های جدولی (Tabular) عملکرد قابل قبول ارائه دهد، و معیارهای ارزیابی استاندارد را تولید کند تا بتوان پیشرفت مدل‌های بعدی را دقیقاً سنجید. بنابراین، در این فاز از الگوریتم Random Forest به عنوان مدل پایه استفاده شد.

دلایل انتخاب الگوریتم Random Forest :

لگوریتم Random Forest به دلایل زیر گزینه‌ی مناسبی برای Baseline در مسئله‌ی تشخیص نفوذ (Intrusion Detection) است:

1.

عملکرد قوی روی داده‌های جدولی و ترکیبیدیتاست پروژه ترکیبی از ویژگی‌های عددی و دسته‌ای (بعد از encoding) است. Random Forest معمولاً روی چنین داده‌هایی عملکرد خوبی دارد و به‌خصوص برای مسائل امنیتی که الگوها غیرخطی هستند مناسب است.

2.

توانایی مدل‌سازی روابط غیرخطی در تشخیص نفوذ، رفتارهای حمله معمولاً با قوانین ساده و خطی قابل تفکیک نیستند (مثلاً تعامل بین تعداد تلاش‌های ورود و زمان دسترسی و Random For- reputation IP). est بدون نیاز به feature engineering پیچیده می‌تواند این روابط غیرخطی را یاد بگیرد.

3.

پایداری در برابر نویز و نقاط پرت داده‌های امنیتی و ترافیک شبکه معمولاً شامل نویز و مقادیر غیرعادی هستند. Random Forest به دلیل ساختار ensemble و میانگین‌گیری بین چندین درخت، نسبت به نویز مقاوم‌تر از مدل‌های ساده است.

4.

نیاز کمتر به تنظیمات و پیش‌پردازش برخلاف بسیاری از مدل‌ها، Random Forest وابستگی زیادی به scaling ندارد و با تنظیمات اولیه (مانند تعداد درخت‌ها و random_state) می‌تواند baseline قابل قبولی تولید کند. این ویژگی برای فاز اول که تمرکز روی ساخت مسیر pipeline و خروجی‌های استاندارد است، بسیار مهم است.

5.

قابل تفسیرتر از برخی مدل‌های پیشرفته Random Forest امکان استخراج feature importance را فراهم می‌کند و می‌تواند در تحلیل‌های بعدی کمک کند که بفهمیم کدام ویژگی‌ها بیشترین اثر را در تشخیص حمله دارند.

نحوه آموزش و ارزیابی Baseline :

پس از آماده‌سازی داده‌ها در مرحله preprocessing و استفاده از داده‌های split شده (Train/Validation/Test)، مدل Random Forest روی بخش Train آموزش داده شد و سپس روی Validation و Test ارزیابی شد. در این baseline برای بهبود رفتار مدل در شرایط احتمالی عدم توازن کلاس‌ها، از تنظیم زیر استفاده شد: `class_weight="balanced"` همچنین برای تکرارپذیری نتایج: `random_state` ثابت در نظر گرفته شد.

معیارهای ارزیابی و خروجی‌ها :

برای ارزیابی عملکرد مدل پایه، از معیارهای استاندارد طبقه‌بندی دودویی استفاده شد:

Accuracy

Precision

Recall

F1-score

Confusion Matrix

AUC-ROC و ROC Curve

این معیارها علاوه بر نمایش عددی، به صورت نمودار نیز ذخیره شدند تا امکان مقایسه دقیق‌تر با مدل‌های پیشرفته‌تر در فاز دوم فراهم شود.

نقش Baseline در ادامه پروژه :

نتایج Random Forest به عنوان خط مبنا در نظر گرفته می‌شود؛ به این معنی که هر مدل پیشرفته‌تر (مانند XGBoost) باید حداقل در معیارهای اصلی پروژه (به‌خصوص Recall و F1-score) عملکرد بهتر یا قابل دفاع‌تری نسبت به این مدل پایه ارائه دهد.

به طور کلی، Random Forest در فاز اول به عنوان یک baseline سریع، پایدار و قابل اعتماد انتخاب شد تا: مسیر اجرای پروژه کامل شود،

معیارها و خروجی‌های استاندارد تولید شوند،

و نقطه‌ای مرجع مشخصی برای مقایسه‌ی مدل‌های فاز دوم وجود داشته باشد.

خروجی های های baseline :

